Pydantic x Modal / October 2026

Why even have an SDK?

Samuel Colvin · Pydantic

whoami - Samuel Colvin

- Creator of **Pydantic**
- Founder & CEO of **Pydantic Labs**

The Pydantic Stack

- Pydantic Validation: contracts for data
- Pydantic AI: production grade agent framework
- Pydantic Logfire: observability, evals, prompt management, LLMOps
- Pydantic AI Gateway: one endpoint, every model
- Pydantic Monty: Python Sandbox

OpenTelemetry already exists. Coding agents exist.

So why did we write three SDKs and a CLI?

A great SDK gets out of the way

You should not have to learn it to use it

Three lines

```python
import logging
import os

from opentelemetry import metrics, trace
from opentelemetry._logs import set_logger_provider
from opentelemetry.exporter.otlp.proto.http import metric_exporter, trace_exporter
from opentelemetry.exporter.otlp.proto.http._log_exporter import OTLPLogExporter
from opentelemetry.instrumentation.fastapi import FastAPIInstrumentor
from opentelemetry.instrumentation.httpx import HTTPXClientInstrumentor
from opentelemetry.sdk._logs import LoggerProvider, LoggingHandler
from opentelemetry.sdk._logs.export import BatchLogRecordProcessor
from opentelemetry.sdk.metrics import MeterProvider
from opentelemetry.sdk.metrics.export import PeriodicExportingMetricReader
from opentelemetry.sdk.resources import Resource
from opentelemetry.sdk.trace import TracerProvider
from opentelemetry.sdk.trace.export import BatchSpanProcessor

resource = Resource.create({
    'service.name': 'weather-app',
    'service.version': '1.0.0',
    'deployment.environment.name': 'prod',
})
base = 'https://logfire-us.pydantic.dev/v1'
headers = {'Authorization': os.environ['LOGFIRE_TOKEN']}

tracer_provider = TracerProvider(resource=resource)
tracer_provider.add_span_processor(
    BatchSpanProcessor(trace_exporter.OTLPSpanExporter(endpoint=f'{base}/traces', headers=headers))
)
trace.set_tracer_provider(tracer_provider)

meter_provider = MeterProvider(
    resource=resource,
    metric_readers=[
        PeriodicExportingMetricReader(
            metric_exporter.OTLPMetricExporter(endpoint=f'{base}/metrics', headers=headers)
        )
    ],
)
metrics.set_meter_provider(meter_provider)
```

```python
import logfire

logfire.configure(service_name='weather-app', environment='prod')
logfire.instrument_fastapi(app, capture_headers=True, excluded_urls='health')
logfire.instrument_httpx()
```

- Same traces, metrics and logs
- The token comes from `.logfire/`, `LOGFIRE_TOKEN`, or an argument
- Header capture and scrubbing are on with one flag; the raw setup needs three lists and still misses your own secrets
- The left side continues off the bottom of the slide

Out of the way, in every language

```python
import logfire

logfire.configure()
logfire.instrument_fastapi(app)
logfire.info('hello {name}', name=user)
```

```typescript
import * as logfire from 'logfire'

logfire.configure()
logfire.info('hello {name}', { name })
```

```rust
let lf = logfire::configure().finish()?;
let _guard = lf.shutdown_guard();
logfire::info!("hello {name}", name = name);
```

- 34 `instrument_*` one-liners in Python: Pydantic AI, OpenAI, Anthropic, httpx, asyncpg, Django, Celery, Redis, MCP, Monty, …
- Each one calls the OpenTelemetry instrumentation package for that library; instrumentation you already have keeps working
- JS wraps the OTel Node SDK and its auto-instrumentations; Rust is a `tracing` layer, so existing `tracing` spans are exported unchanged
- `send_to_logfire=False` and it exports to any OTLP backend

A great SDK is the agent's interface too

One binary, multiple ecosystem

```
➤ uvx logfire
➤ npx logfire
```

- One Rust binary. PyPI wheels for 17 targets, npm for 8. Neither Rust nor Python is needed to run it
- **For people:** `logfire signup`, `auth`, `init`, `project current`, `token read`
- **For scripts:** `--output json|jsonl`, `--no-input`, documented exit codes
- **For agents:** `logfire wizard` starts Claude Code or Codex in your repo with the Logfire setup skill; `logfire mcp ...` calls the hosted MCP server

Agent-friendly CLI

A person, stdin is a terminal

```
$ logfire init
? Select a project
❯ samuelcolvin/demo
  samuelcolvin/dev
  samuelcolvin/gateway
```

Arrow keys, a select menu, a browser opens for login.

An agent, stdin is a pipe

```
$ logfire init
Error: not a terminal: run `logfire init new` or
`logfire init use` (with --name <project-name>)

$ logfire init use --name demo --permission send
```

No menu to get stuck in. The error names the arguments, the agent passes them and retries.

- The same binary decides per invocation: `isatty(stdin)`
- `--no-input` forces the second mode, `--output json` gives the agent a stable shape to parse

Prompts reach the agent through the same SDK

```python
prompt = logfire.template_var(
    name='prompt__weather_agent',
    type=str,
    default='Combine tool calls into a single code execution call. Answer in {{units}}.',
    inputs_type=WeatherInputs,  # a BaseModel with `units: str`
)

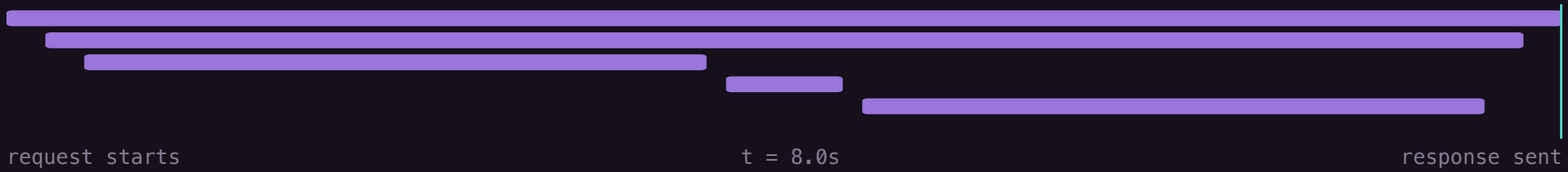
with prompt.get(WeatherInputs(units='celsius'), label='production') as resolved:
    result = await weather_agent.run(question, instructions=resolved.value)
```

- A prompt is a managed variable named `prompt__<slug>`, with the same versions, targeting rules and rollouts as a feature flag
- No second vendor SDK: the import that sends traces also fetches prompts
- Other languages: Logfire serves **OFREP**, so any **OpenFeature** client can evaluate the same variables

A great SDK does what the protocol can't

OTLP carries finished spans. Your users are waiting on unfinished ones.

Pending spans



PLAIN OTLP: SPANS ARRIVE WHEN THEY END

- chat gpt-5.6 3.2s
- run_code 0.6s
- chat gpt-5.6 3.2s
- weather_agent run 7.6s
- GET /weather 8.0s

WITH PENDING SPANS: ROWS APPEAR WHEN THEY START

- GET /weather 8.0s
- weather_agent run 7.6s
- chat gpt-5.6 3.2s
- run_code 0.6s
- chat gpt-5.6 3.2s

- When a span starts, the SDK exports a zero-duration copy tagged `pending_span`; the backend replaces it when the real span arrives
- The copy is dropped if the real span is in the same export batch, so short spans are sent once

Values that look like your code

What an OTLP attribute can hold

```
"{\"id\": 1, \"email\": \"x@y.z\",
  \"signed_up\": \"2026-01-01T09:30:00Z\"}"
```

A string, a number, a bool, or a flat list of those.

What Logfire shows

```
User(
    id=1,
    email='x@y.z',
    signed_up=datetime(2026, 1, 1, 9, 30),
)
```

The SDK sends a small schema next to the value, and the UI renders it as the Python you wrote.

- Works for Pydantic models, dataclasses, `datetime`, `Decimal`, `UUID`, `Enum`, `Path`, exceptions, DataFrames
- Other backends still receive the plain attribute

Demo

🤞

Thank you

Thanks to **Modal** for hosting

pydantic.dev/contact

`@samuelcolvin` · `@pydantic`